HW #4.3: Reproducible Data Analysis

Jooan Choi

Table of contents

1 Introduction 1

2 Busiest Airport 1

3 Monte Carlo Numerical Simulation 2

4 Planning and Prompting Gen AI 3

5 Reflection 5

6 Appendix A: GenAI Usage 5

7 Appendix B: Code Appendix 7

1 Introduction

This report presents a reproducible data analysis using Quarto, including an airport traffic analysis, a Monte Carlo simulation, and an AI prompting comparison. The goal is to clearly communicate both the results and the analytical process.

2 Busiest Airport

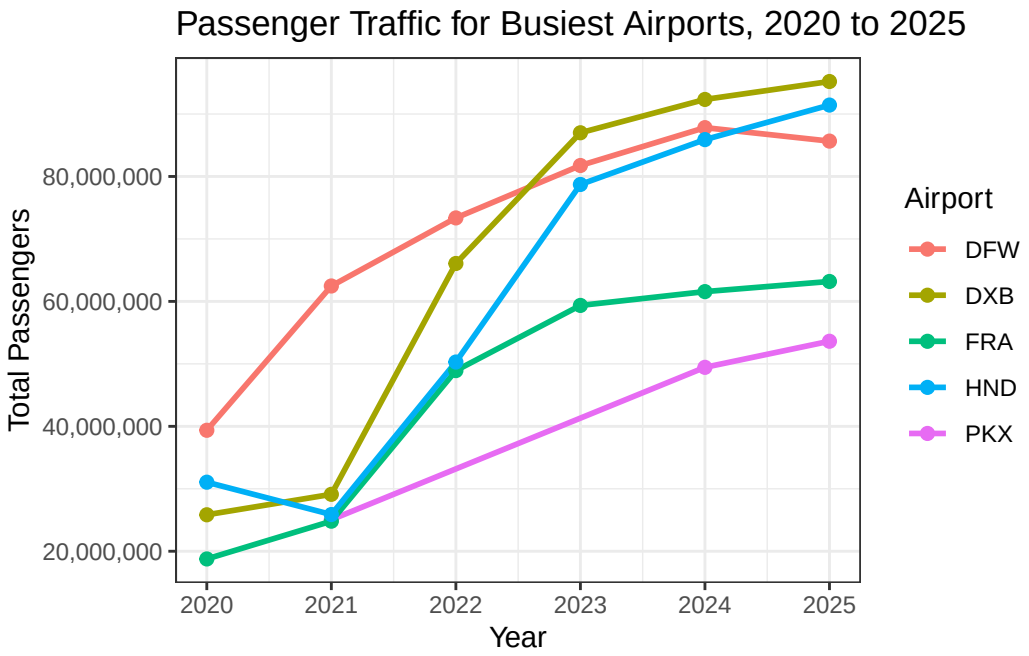
In this section, I analyzed passenger traffic for major airports from 2020 to 2025. After collecting the data and cleaning the variables, I created both a table and a line graph to show how passenger totals changed over time.

The table shows passenger traffic for five major airports from 2020 to 2025. Across all airports, passenger numbers generally increase over time, indicating recovery after lower levels in 2020. Airports such as DFW and DXB show particularly strong growth, while others like HND and FRA also display steady increases.

Table 1: Passenger Traffic for Busiest Airports, 2020 to 2025

Airport	IATA	ICAO	2020	2021	2022	2023	2024	2025
Beijing Daxing International Airport	PKX	ZBAD	25,051,012	49,441,029	53,618,949	NA	NA	NA
Dallas Fort Worth International Airport	DFW	KDFW	62,465,756	87,817,864	85,660,127	39,364,990	73,362,946	81,755,538
Dubai International Airport	DXB	OMDB	29,110,609	92,331,506	95,200,000	25,836,771	66,069,981	86,994,365
Frankfurt Airport	FRA	EDDF	24,812,849	61,564,957	63,189,666	18,768,601	48,918,482	59,355,389
Tokyo Haneda Airport	HND	RJTT	25,876,429	85,900,167	91,430,000	31,055,210	50,290,705	78,719,302

Figure 1: Passenger traffic trends for major airports from 2020 to 2025

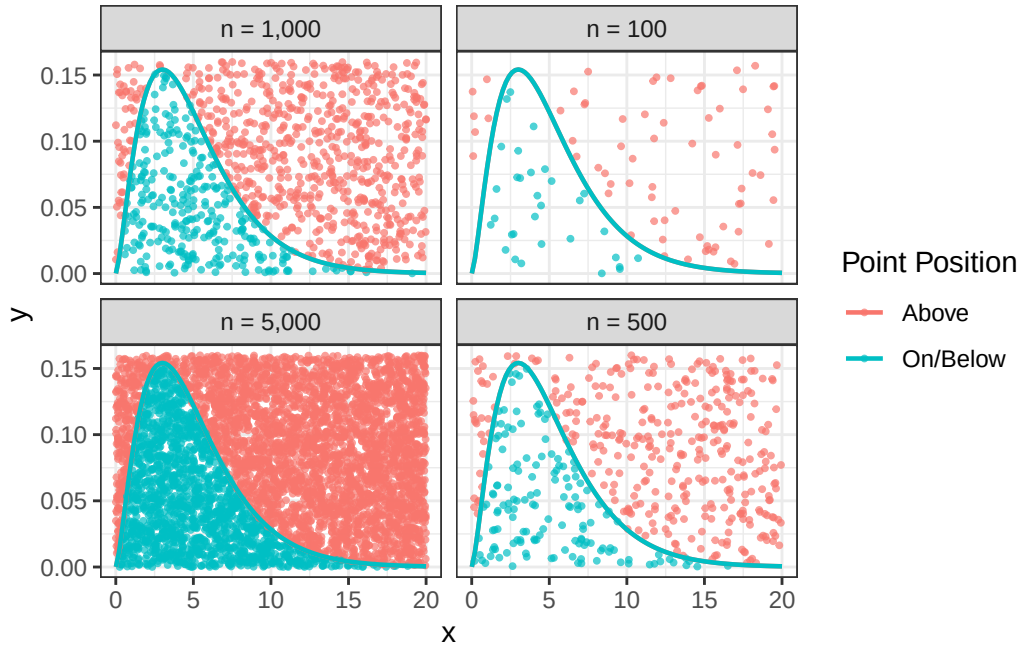


The table and graph show passenger traffic trends for major airports from 2020 to 2025. All airports experience increasing passenger counts after 2020, indicating recovery over time. DXB and HND show the strongest growth, while FRA and PKX increase more gradually. The table provides exact values, while the graph highlights overall patterns and differences across airports.

3 Monte Carlo Numerical Simulation

Next, I use Monte Carlo simulation to estimate the area under a chi-square density curve with 5 degrees of freedom from x = 0 to 20.

Figure 2: Monte Carlo small multiple plots showing how larger sample sizes improve the estimate



The small multiple plots show that the Monte Carlo estimate becomes more stable as the number of simulated points increases. With fewer points, the estimate is more affected by random variation, while larger samples produce a denser and more consistent approximation of the curve.

The exact value of the integral is approximately one, since most of the chi-square density lies between 0 and 20. The higher plots support this by showing a more stable proportion of points below the curve.

4 Planning and Prompting Gen AI

My plan was to first reshape the dataset from wide format into long format so that each row would represent one observation at one time point. I then planned to create variables for treatment group and year, which would make the data easier to visualize and analyze. After tidying the data, I would create a line plot comparing calcium trends over time for the two groups.

I used ChatGPT for this task. I uploaded the original csv file and used a prompt that explained my plan for tidying and visualizing the data.

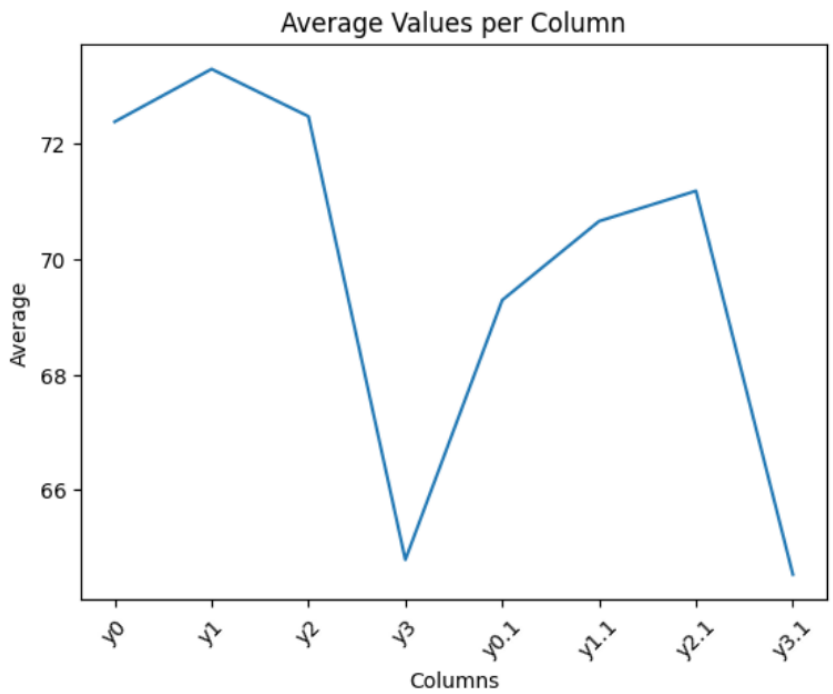
Prompt: "Please check the uploaded calcium.csv file. The data are in wide format, with calcium measurements across four time points for two groups stored in separate columns. I want to reshape the data into tidy format, create variables for treatment group and year, and then generate R code for a line plot comparing calcium trends over time between the two groups. Please explain the steps and provide the R code."

Response: The plan-based response provided clear steps for reshaping the dataset using pivoting techniques, separating the two groups, and creating variables for group and time. The response also

included R code to convert the data into long format and generate a line plot comparing calcium levels across time between the groups.

Because the prompt included a detailed plan, the output closely matched the structure of the dataset and produced a visualization that clearly showed differences in trends between the groups.

Generic response:



I created a visualization showing the **average value of each column** in your dataset.

What this shows:

- Each point represents the **mean (average)** of one column (y0, y1, y2, y3, etc.).
- You can quickly compare which variables tend to have higher or lower values overall.
- For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create:

- Boxplots (to show spread and outliers)
- Time series plots (if rows represent time/order
- Correlation heatmaps
- Side-by-side comparisons (e.g., y0 vs y0.1)

Just tell me

Prompt: What is your version information?

You're chatting with **ChatGPT based on the GPT-5.3 architecture**.

Key details:

- **Model**: GPT-5.3
- **Knowledge cutoff**: August 2025
- **Current date (system)**: March 31, 2026

If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know

(This chat took place on Tuesday, March 31st, 2026.)

Comparison: Comparing the two approaches, the plan-based prompt produced a more useful result because it reflected the structure of the dataset and focused on comparing trends over time between the two groups.

In contrast, the generic prompt only created a plot of average values by column, which did not capture the group or time structure of the data. This shows that a more detailed prompt leads to more meaningful and accurate outputs.

5 Reflection

I learned how to create a reproducible analysis using quarto by combining code, visualizations, and narrative text. I also saw how restructuring data into tidy format makes analysis and visualization much easier. Finally, I learned that well planned prompts lead to more concise AI outputs compared to generic prompts.

6 Appendix A: GenAI Usage

Prompt 1: Please check the uploaded calcium.csv file. The data are in wide format, with calcium measurements across four time points for two groups stored in separate columns. I want to reshape the data into tidy format, create variables for treatment group and year, and then generate R code for a line plot comparing calcium trends over time between the two groups. Please explain the steps and provide the R code.

Prompt 2: What is your version?

Response 1: The plan-based response provided clear steps for reshaping the dataset using pivoting techniques, separating the two groups, and creating variables for group and time. The response also included R code to convert the data into long format and generate a line plot comparing calcium levels across time between the groups.

Response 2: You're chatting with ChatGPT based on the GPT-5.3 architecture.

Moreover, the table exceeded the page width when rendered to PDF due to long airport names and multiple columns. To resolve this issue, I used AI assistance to modify the table formatting so that it fits within the page.

Model: GPT-5.3 Knowledge cutoff: August 2025 Current date (system): March 31, 2026

Moreover, The original table exceeded the page width in the PDF output. After revising the formatting, the table was adjusted to fit properly within the page.

Original code:

```
# Step 7: Render Table
airportTable |>
  kable(
    caption = "Passenger Traffic for Busiest Airports, 2020 to 2025",
    col.names = c("Airport", "IATA", "ICAO", "2020", "2021", "2022", "2023", "2024", "2025")
  ) |>
  kable_classic(
    lightable_options = "striped"
  )
```

Table 2: Passenger Traffic for Busiest Airports, 2020 to 2025

Airport	IATA	ICAO	2020	2021	2022	2023
Beijing Daxing International Airport	PKX	ZBAD	25,051,012	49,441,029	53,618,949	NA
Dallas Fort Worth International Airport	DFW	KDFW	62,465,756	87,817,864	85,660,127	39,364,990
Dubai International Airport	DXB	OMDB	29,110,609	92,331,506	95,200,000	25,836,771
Frankfurt Airport	FRA	EDDF	24,812,849	61,564,957	63,189,666	18,768,601
Tokyo Haneda Airport	HND	RJTT	25,876,429	85,900,167	91,430,000	31,055,210

Revised code (Written by GPT-5.3):

```
airportTable |>
  kable(
    caption = "Passenger Traffic for Busiest Airports, 2020 to 2025",
    col.names = c("Airport", "IATA", "ICAO", "2020", "2021", "2022", "2023", "2024", "2025"),
    booktabs = TRUE,
    format = "latex",
    escape = FALSE
  ) |>
  kable_styling(
    latex_options = c("hold_position", "scale_down"),
    font_size = 7
  ) |>
  column_spec(1, width = "10em")
```

Table 3: Passenger Traffic for Busiest Airports, 2020 to 2025

Airport	IATA	ICAO	2020	2021	2022	2023	2024	2025
Beijing Daxing International Airport	PKX	ZBAD	25,051,012	49,441,029	53,618,949	NA	NA	NA
Dallas Fort Worth International Airport	DFW	KDFW	62,465,756	87,817,864	85,660,127	39,364,990	73,362,946	81,755,538
Dubai International Airport	DXB	OMDB	29,110,609	92,331,506	95,200,000	25,836,771	66,069,981	86,994,365
Frankfurt Airport	FRA	EDDF	24,812,849	61,564,957	63,189,666	18,768,601	48,918,482	59,355,389
Tokyo Haneda Airport	HND	RJTT	25,876,429	85,900,167	91,430,000	31,055,210	50,290,705	78,719,302

7 Appendix B: Code Appendix

```r
# Busiest Airports Data Wrangling ----
## Passenger traffic data for airports
## from 2020 through 2025.

# Step 1: Load Packages ----
## Needed Packages: ggplot2, tidyverse, rvest
library(ggplot2)
library(tidyverse)
library(rvest)

# Step 2: Load Data ----
wikiURL <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"

airportRawList <- read_html(x = wikiURL) |>
  html_elements(css = "table.wikitable") |>
  html_table()

# Step 3: Select the Needed Tables ----
airport2025 <- airportRawList[[1]] |> mutate(year = 2025)
airport2024 <- airportRawList[[2]] |> mutate(year = 2024)
airport2023 <- airportRawList[[3]] |> mutate(year = 2023)
airport2022 <- airportRawList[[4]] |> mutate(year = 2022)
airport2021 <- airportRawList[[5]] |> mutate(year = 2021)
airport2020 <- airportRawList[[6]] |> mutate(year = 2020)

# Step 4: Combine the Yearly Tables
airportTrafficRaw <- bind_rows(
  airport2025,
  airport2024,
  airport2023,
  airport2022,
  airport2021,
```

```r
  airport2020
)

# Step 5: Keep and Clean Needed Variables
airportTraffic <- airportTrafficRaw |>
  mutate(
    Code_IATA_ICAO = `Code(IATA/ICAO)`,
    Total_passengers = parse_number(Totalpassengers)
  ) |>
  select(
    year,
    Airport,
    Code_IATA_ICAO,
    Total_passengers
  ) |>
  separate_wider_delim(
    cols = Code_IATA_ICAO,
    delim = "/",
    names = c("IATA", "ICAO")
  ) |>
  filter(
    Airport %in% c(
      "Hartsfield-Jackson Atlanta International Airport",
      "Frankfurt Airport",
      "Beijing Daxing International Airport",
      "Dubai International Airport",
      "Tokyo Haneda Airport",
      "Dallas Fort Worth International Airport"
    )
  ) |>
  arrange(Airport, year)

library(knitr)
library(kableExtra)

# Step 6: Create Table Data
airportTable <- airportTraffic |>
  mutate(
    Total_passengers = scales::comma(Total_passengers)
  ) |>
  pivot_wider(
    names_from = year,
    values_from = Total_passengers
  ) |>
  arrange(Airport)

# Step 7: Render Table
```

```r
airportTable |>
  kable(
    caption = "Passenger Traffic for Busiest Airports, 2020 to 2025",
    col.names = c("Airport", "IATA", "ICAO", "2020", "2021", "2022", "2023", "2024", "2025")
  ) |>
  kable_classic(
    lightable_options = "striped"
  )

# Step 8: Create Plot
ggplot(
  data = airportTraffic,
  mapping = aes(
    x = year,
    y = Total_passengers,
    color = IATA,
    group = Airport
  )
) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_continuous(
    labels = scales::comma
  ) +
  scale_x_continuous(
    breaks = 2020:2025
  ) +
  labs(
    title = "Passenger Traffic for Busiest Airports, 2020 to 2025",
    x = "Year",
    y = "Total Passengers",
    color = "Airport"
  ) +
  theme_bw()

# Monte Carlo Simulation
monte <- function(n_points, x_min, x_max, y_min, y_max, fun) {
  data.frame(
    x = runif(n_points, min = x_min, max = x_max),
    y = runif(n_points, min = y_min, max = y_max)
  ) |>
    mutate(
      curve_y = fun(x),
      position = if_else(y <= curve_y, "On/Below", "Above")
    )
}
```

```r
set.seed(184)

sim_100 <- monte(100, 0, 20, 0, 0.16, function(x) dchisq(x, df = 5)) |>
  mutate(n_label = "n = 100")

sim_500 <- monte(500, 0, 20, 0, 0.16, function(x) dchisq(x, df = 5)) |>
  mutate(n_label = "n = 500")

sim_1000 <- monte(1000, 0, 20, 0, 0.16, function(x) dchisq(x, df = 5)) |>
  mutate(n_label = "n = 1,000")

sim_5000 <- monte(5000, 0, 20, 0, 0.16, function(x) dchisq(x, df = 5)) |>
  mutate(n_label = "n = 5,000")

mc_all <- bind_rows(sim_100, sim_500, sim_1000, sim_5000)

ggplot(mc_all, aes(x = x, y = y, color = position)) +
  geom_point(size = 0.7, alpha = 0.7) +
  stat_function(fun = dchisq, args = list(df = 5), linewidth = 0.8) +
  coord_cartesian(xlim = c(0, 20), ylim = c(0, 0.16)) +
  facet_wrap(~ n_label, ncol = 2) +
  labs(
    x = "x",
    y = "y",
    color = "Point Position"
  ) +
  theme_bw()
```